**Name:Antoine Sobhy Soliman**
**ID:23010303**

**GIT HUB REPO:**

**https://github.com/antoine174/Sorting_Algorithms_Visualization.git**

# DSA LAB1

## 1. Design and Code Structure

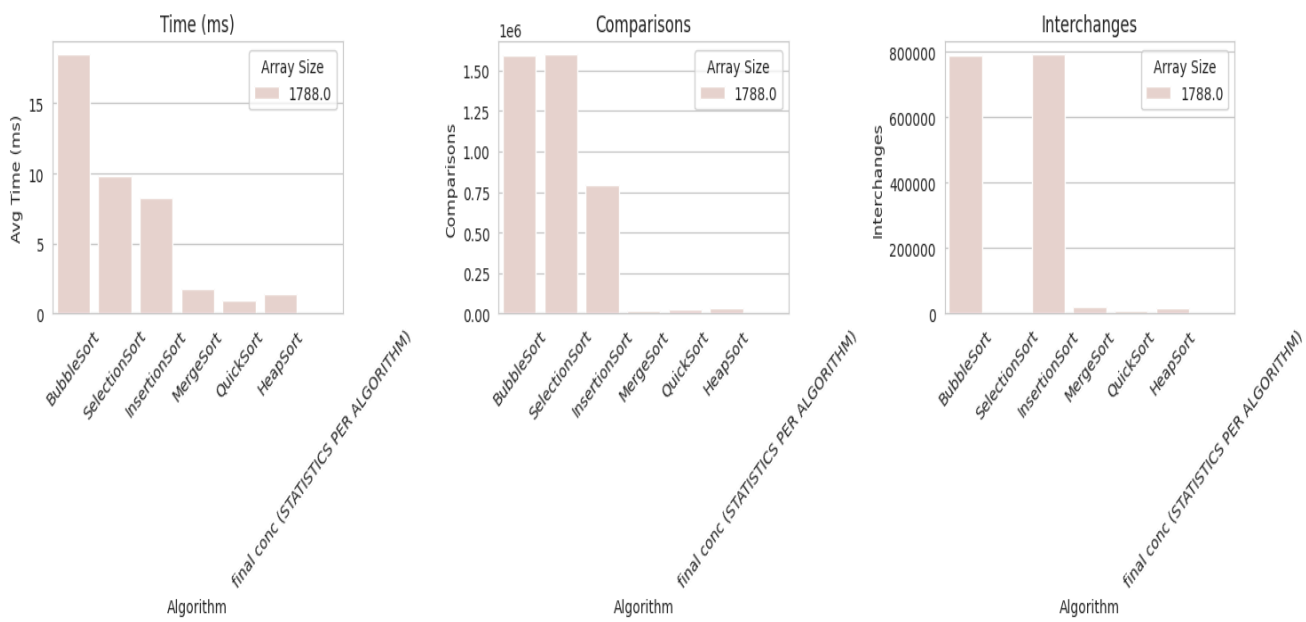
- **interfaces & algorithms:** Contains the Sorts interface and the sorting algorithms (Bubble, Selection, Insertion, Merge, Quick, Heap). They all extend an AbstractSort base class.
- **ui:** Contains the VisualizerApp, which is the JavaFX user interface. It has two tabs: one for visualizing the sorts and one for running batch comparisons.
- **utils:** Contains helper classes like ArrayGenerate (to create or load arrays) and BatchTestRunner / ComparisonEngine (to handle the background benchmarking and CSV exporting).

## 2. Design Decisions

- **Using an Abstract Class:** I used the AbstractSort class to handle common tasks like tracking comparisons, tracking interchanges, and managing the UI delay.
- **Background Threading:** I ran the sorting algorithms on a separate background thread  If I ran them on the main thread, the JavaFX UI would freeze entirely during the sorting process. Platform.runLater() is used to safely update the UI bars mid-sort.
- **Separating Benchmarks from the UI:** I built a separate ComparisonEngine that runs without any visual updates. This ensures that the time measurements are accurate and not slowed down by GUI drawing times.
- **the visualization:** at each time i use increment change function i give the bars each new height which mean i loop all over them again which is simple implementation for n<=100 (which satisfy the given)

# 3. Sorting Comparison Output and Analysis

| | A | B | C | D | E | F | G | H | I | J |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | Algorithm | Array Size | Generation Mode | Runs | Avg Time (ms) | Min Time (ms) | Max Time (ms) | Comparisons | Interchanges | |
| 2 | BubbleSort | 1788 | test3.csv (Row 1) | 5 | 18.4766 | 10.0546 | 40.3715 | 1597082 | 790021 | |
| 3 | SelectionSort | 1788 | test3.csv (Row 1) | 5 | 9.7969 | 2.5659 | 23.2303 | 1597578 | 1787 | |
| 4 | InsertionSort | 1788 | test3.csv (Row 1) | 5 | 8.2834 | 1.4042 | 22.5081 | 791805 | 791805 | |
| 5 | MergeSort | 1788 | test3.csv (Row 1) | 5 | 1.7786 | 1.161 | 4.0996 | 17453 | 19408 | |
| 6 | QuickSort | 1788 | test3.csv (Row 1) | 5 | 0.9283 | 0.6648 | 1.9401 | 22509 | 10317 | |
| 7 | HeapSort | 1788 | test3.csv (Row 1) | 5 | 1.3684 | 1.3041 | 1.514 | 37484 | 17848 | |
| 8 | | | | | | | | | | |
| 9 | final conc (STATISTICS PER ALGORITHM) | | | | | | | | | |
| 10 | Algorithm | Total Cases Tested | Generation Mode | - | Overall Avg Time | Absolute Min Time | Absolute Max Time | Avg Comparisons | Avg Interchanges | |
| 11 | BubbleSort SUMMARY | 1 | ALL ROWS COMBINED | - | 18.4766 | 10.0546 | 40.3715 | 1597082 | 790021 | |
| 12 | SelectionSort SUMMARY | 1 | ALL ROWS COMBINED | - | 9.7969 | 2.5659 | 23.2303 | 1597578 | 1787 | |
| 13 | InsertionSort SUMMARY | 1 | ALL ROWS COMBINED | - | 8.2834 | 1.4042 | 22.5081 | 791805 | 791805 | |
| 14 | MergeSort SUMMARY | 1 | ALL ROWS COMBINED | - | 1.7786 | 1.161 | 4.0996 | 17453 | 19408 | |
| 15 | QuickSort SUMMARY | 1 | ALL ROWS COMBINED | - | 0.9283 | 0.6648 | 1.9401 | 22509 | 10317 | |
| 16 | HeapSort SUMMARY | 1 | ALL ROWS COMBINED | - | 1.3684 | 1.3041 | 1.514 | 37484 | 17848 | |
| 17 | | | | | | | | | | |




- 

**Analysis:**

Based on the batch test results, the algorithms clearly split into two performance groups:

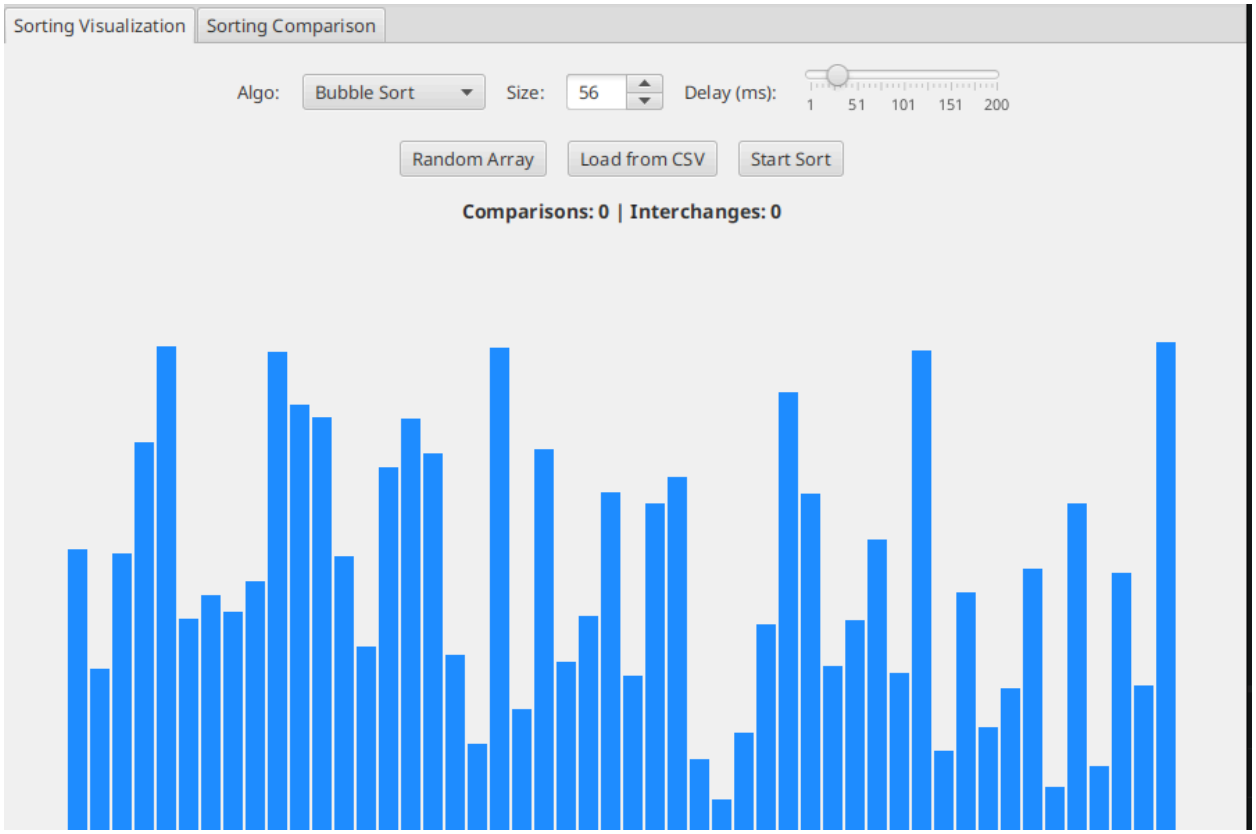
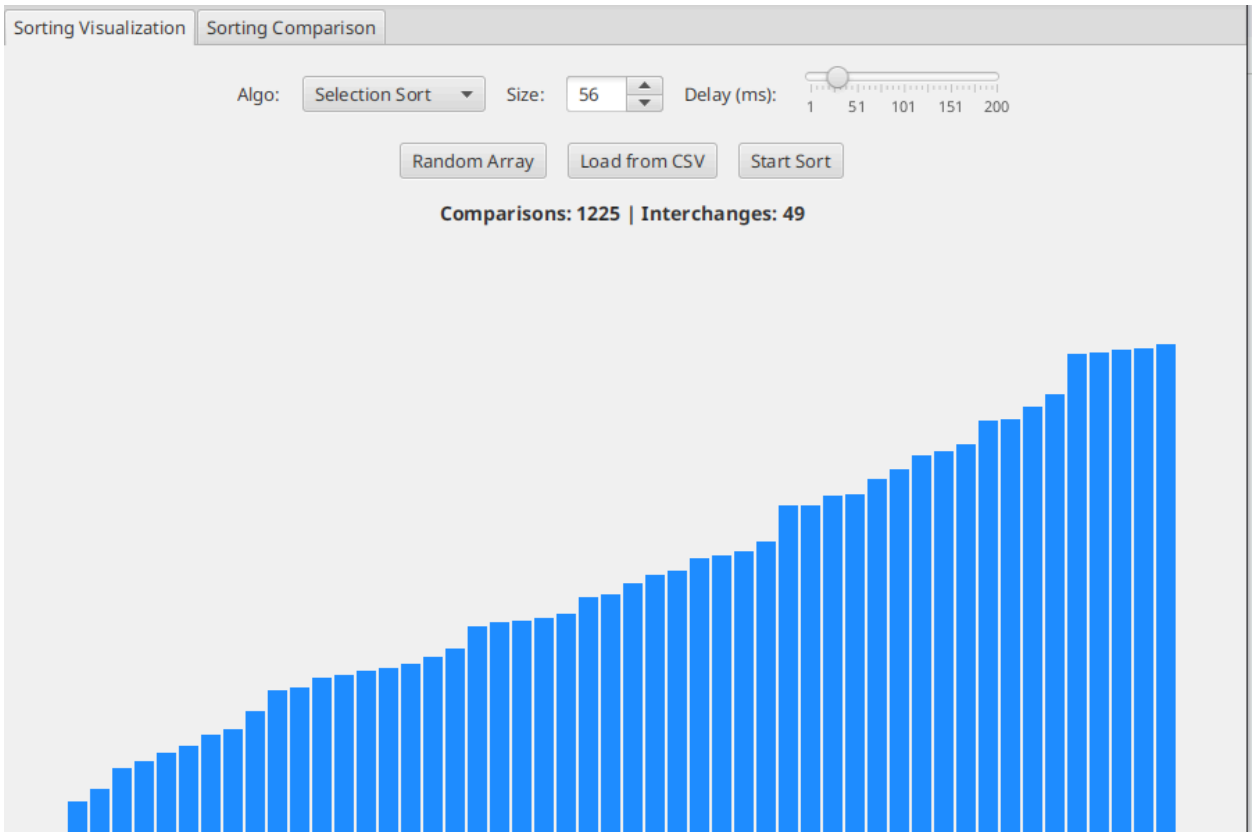
- **O(N^2) Algorithms (Bubble, Selection, Insertion):** These were very slow on larger arrays. Bubble sort was consistently the slowest due to the high number of swaps. Insertion sort performed poorly on inversely sorted data but was extremely fast on

already-sorted data.

- **O(N log N) Algorithms (Merge, Quick, Heap):** These algorithms handled large arrays effortlessly. Quick Sort generally had the best average time on random arrays, while Merge Sort was consistent across all data types but used more memory.

# 4. Sample Outputs for Sorting Visualization

- **Screenshot 1:** The initial random array generated in the UI.



- **Screenshot 2:** after sort